



Building Grafana dashboards to visualize Prometheus metrics

What is Prometheus

It was originally developed at SoundCloud, by ex-Googlers. But its story goes beyond that, Google had a dynamic container based system for all their systems in production called Borg and was using an in house monitoring solution called Borgmon for the same. But in SoundCloud there were no alternative for the same, other solutions at the time doesn't fit the need of the dynamic environments (similar to Kubernetes). So, the inception of Prometheus began building an open source solution on the same key principles for the similar kind of environment. I think the Author's sentiment captures it the best

"They are twins separated at birth." Kubernetes directly builds on Google's decade-long experience with their own cluster scheduling system, Borg. Prometheus's bonds to Google are way looser but it draws a lot of inspiration from Borgmon, the internal monitoring system Google came up with at about the same time as Borg. In a very sloppy comparison, you could say that Kubernetes is Borg for mere mortals, while Prometheus is Borgmon for mere mortals. Both are "second systems" trying to iterate on the good parts while avoiding the mistakes and dead ends of their ancestors.

What is Prometheus

So what is it basically, Prometheus is an open-source systems monitoring and alerting toolkit (Yes it's very modular!). It collects metrics from configured targets at given intervals, evaluates rule expressions, displays the results, and can trigger alerts if some condition is observed to be true.

It got adopted by Cloud Native Computing Foundation as the second project to be adopted right after Kubernetes itself. Today, Prometheus is the de facto monitoring tool used alongside Kubernetes everywhere.

Time series Databases

A time series database (TSDB) is a database optimized for time-stamped or time series data. Time series data are simply measurements or events that are tracked, monitored, downsampled, and aggregated over time. This could be server metrics, application performance monitoring, network data, sensor data, events, clicks, trades in a market, and many other types of analytics data. (Internally it uses 64bit floating point XOR based compression algorithm that Prometheus uses under the hood)

A time series database is built specifically for handling metrics and events or measurements that are time-stamped. A TSDB is optimized for measuring change over time. Properties that make time series data very different than other data workloads are data lifecycle management, summarization, and large range scans of many records.

Time series Databases

If you look up most popular TSDBs right now, you will find, Prometheus to be in one of the top ones.

1. InfluxDB
2. Kdb+
3. Prometheus
4. Graphite
5. RRDTOol

6. Druid
7. OpenTSDB
8. TimescaleDB
9. FaunaDB
10. KairosDB

11. GridDB
12. Alibaba Cloud TSDB
13. DolphinDB
14. eXtremeDB
15. Amazon Timestream

Why is a time series database important now?

Time series databases are not new, but the first-generation time series databases were primarily focused on looking at financial data, the volatility of stock trading, and systems built to solve trading. But its scope is no longer limited to just that. The fundamental conditions of computing have changed dramatically over the last decade. Everything has become compartmentalized. Monolithic mainframes have vanished, replaced by serverless servers, micro serverices, and containers. With these kind of architecture, finding the sequence of things happening in real time over many many systems running in parallel requires a monitoring system build on top of solid TSDB foundation.

Push vs Pull Model for Metrics Collection

Metrics are **one of the “go-to” standards** for any monitoring system of which there are a variety of different types. At its core, **a metric is essentially a measurement of a property of a portion of an application or system**. Metrics make an observation by keeping track of the state of an object. These **observations** are some value or a series of related values **combined with a timestamp that describes the observations**, the output of which is commonly **called time-series data**.

The canonical example for metrics gathering is **website hits**, where we regularly collect various data points like **the number of times someone visits** a particular page and **the location or source of the visitor** for a given site. Examples of different types of metrics are as follows:

- Counters
- Timers
- Events
- Logs

Push vs Pull Model for Metrics Collection

A single metric in and of itself is often not that useful, but when visualized over time especially in conjunction with some mathematical transformation, metrics can give great insight into a system and its behaviour. Following are some common transformations for metrics:

- Sum
- Average
- Median
- Percentiles
- Standard deviation
- Rates of change

Push vs Pull Model for Metrics Collection

In Push-based monitoring the application being monitoring becomes an emitter of data “pushing” metrics and events on some time interval or a constraint violation. This approach typically favors a decentralized structure and has the advantage of not requiring the monitoring system to “pre-register” the monitored component as the emitter pushes data to the configured destination upon start. But it suffers from several drawbacks and loses its clear visibility if a push based agent dies or doesn’t respond over time. Also, the amount of data pushed needs be optimized in this approach(because it is a part of the application and application throughput in some sense.)

Pull/polling based solutions works differently in that they pull data which are exposed via different services and endpoints. are quite common in monitoring and historically favors a centralized organizational structure, although this is not a requirement. Primarily, in pull-based monitoring, the monitoring system asks or queries a monitored component, for example, pinging a host or to see if a website is up.

Architecture

Most Prometheus components are written in Go, making them easy to build and deploy as static binaries for all types of target/systems. Prometheus scrapes metrics from instrumented jobs, either directly or via an intermediary push gateway for short-lived jobs. It stores all scraped samples locally and runs rules over this data to either aggregate and record new time series from existing data or generate alerts.

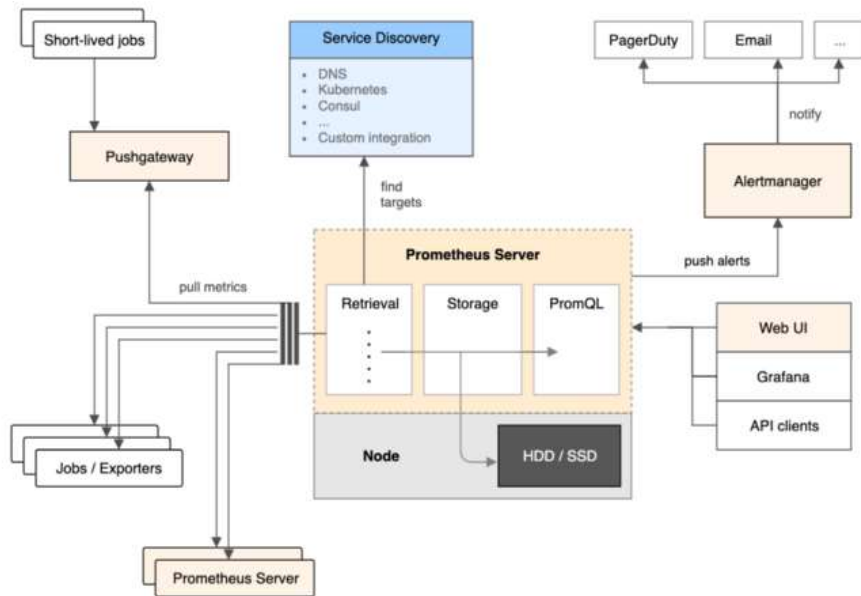
Prometheus's main features are:

- a multi-dimensional data model with time series data identified by metric name and key/value pairs
- PromQL, a flexible query language to leverage this dimensionality
- Time series collection happens via a pull model over HTTP
- pushing time series is supported via an intermediary gateway
- targets are discovered via service discovery or static configuration
- multiple modes of graphing and dashboarding support

Architecture

The Prometheus ecosystem consists of multiple components, many of which are optional:

- the main Prometheus server which scrapes and stores time series data
- client libraries for instrumenting application code
- a push gateway for supporting short-lived jobs
- special-purpose exporters for services like HAProxy, StatsD, Graphite, etc. which send metrics from an existing system or software.
- an alertmanager to handle alerts



When does it fit?

Prometheus works well for recording any purely numeric time series. It fits both machine-centric monitoring as well as monitoring of highly dynamic service-oriented architectures like Kubernetes. In a world of microservices, its support for multi-dimensional data collection and querying is a particular strength.

Prometheus is designed for reliability, to be the system you go to during an outage to allow you to quickly diagnose problems. Each Prometheus server is standalone, not depending on network storage or other remote services. You can rely on it when other parts of your infrastructure are broken, and you do not need to setup extensive infrastructure to use it.

When does it not fit?

Prometheus values reliability. You can always view what statistics are available about your system, even under failure conditions. If you need 100% accuracy, such as for per-request billing, Prometheus is not a good choice as the collected data will likely not be detailed and complete enough. In such a case you would be best off using some other system to collect and analyze the data for billing, and Prometheus for the rest of your monitoring.

Prometheus Setup

For the demonstration purposes we will use Docker to make things really easy and reproducible anywhere. Here is a simple Dockerfile which sets up the stage for us.

- It starts off with Ubuntu 18.04 (bionic) official image
- Installs some of the tools we will be using like `wget`, `screen` and `vim`
- It downloads the latest binary releases for Prometheus, `node_exporter` and `alertmanager` (As we said before, Prometheus is highly modular)
- It also downloads Grafana which will be used later for Visualization
- Exposes the default ports from the respective services

```
FROM ubuntu:bionic
MAINTAINER Arindam Paul, arindampaul1989@gmail.com
```

```
RUN apt-get update \
    && apt-get install -y wget \
    && apt-get install -y screen \
    && apt-get install -y vim
```

```
WORKDIR /root
RUN wget -nv https://github.com/prometheus/prometheus/releases/download/v2.18.0/prometheus-2.18.0.linux-amd64.tar.gz
RUN wget -nv https://github.com/prometheus/node_exporter/releases/download/v0.18.1/node_exporter-0.18.1.linux-amd64.tar.gz
RUN wget -nv https://github.com/prometheus/alertmanager/releases/download/v0.20.0/alertmanager-0.20.0.linux-amd64.tar.gz
RUN wget -nv https://dl.grafana.com/oss/release/grafana-6.7.3.linux-amd64.tar.gz
```

```
# node_exporter
EXPOSE 9100
```

```
# prometheus
EXPOSE 9090
```

```
# grafana
EXPOSE 3000
```

```
# alertmanager
EXPOSE 9093
```

Prometheus Setup

Now, we will create a simple docker compose file to prepare our docker image to run, only thing it does is, it gives us a handy name to work with which will expose multiple ports to the host for us. With this we can access them from host machine. I generally prefer docker compose a lot than writing long docker run commands with options.

```
version: '3'
services:
  prometheus_demo:
    build: .
    ports:
      - "9100:9100"
      - "9090:9090"
      - "3000:3000"
      - "9093:9093"
```

Once these two files are there, you can go to the folder containing them and run the service from docker-compose in interactive mode. Note that `--service-ports` is very important option, it allows us to do the port binding right (which is disabled by default).

```
1 docker-compose run --service-ports prometheus_demo
```

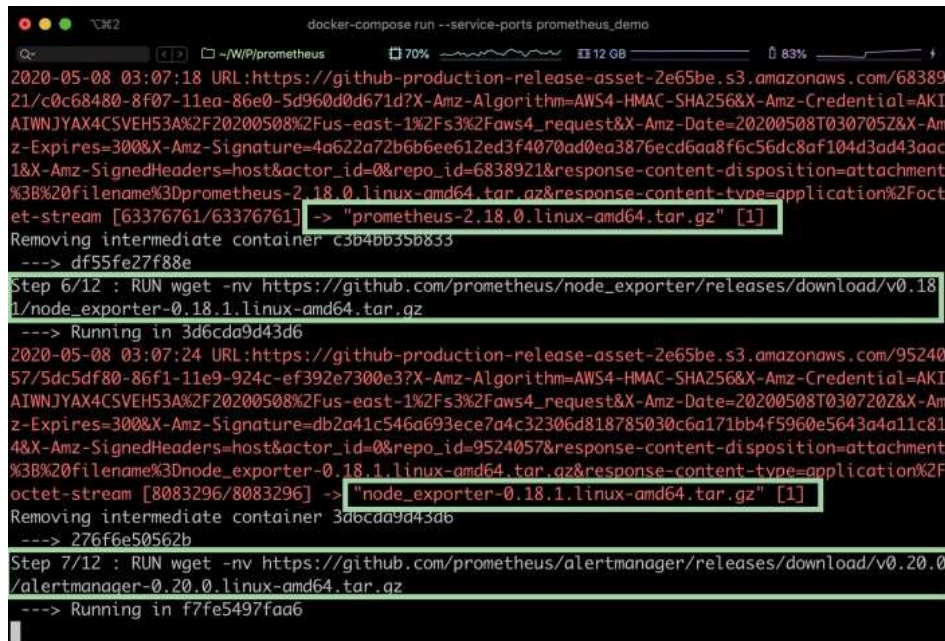
Prometheus Setup

```
docker-compose run --service-ports prometheus_demo

---> c3c304cb4f22
Step 2/12 : MAINTAINER Arindam Paul, arindampaul1989@gmail.com
---> Using cache
---> 4ec8d8f212bc
Step 3/12 : RUN apt-get update && apt-get install -y wget
---> Running in fd8dcd5dc715
Get:1 http://security.ubuntu.com/ubuntu bionic-security InRelease [88.7 kB]
Get:2 http://archive.ubuntu.com/ubuntu bionic InRelease [242 kB]
Get:3 http://security.ubuntu.com/ubuntu bionic-security/main amd64 Packages [908 kB]
Get:4 http://archive.ubuntu.com/ubuntu bionic-updates InRelease [88.7 kB]
Get:5 http://archive.ubuntu.com/ubuntu bionic-backports InRelease [74.6 kB]
Get:6 http://archive.ubuntu.com/ubuntu bionic/universe amd64 Packages [11.3 MB]
Get:7 http://security.ubuntu.com/ubuntu bionic-security/multiverse amd64 Packages [8505 B]
Get:8 http://security.ubuntu.com/ubuntu bionic-security/restricted amd64 Packages [52.4 kB]
Get:9 http://security.ubuntu.com/ubuntu bionic-security/universe amd64 Packages [844 kB]
Get:10 http://archive.ubuntu.com/ubuntu bionic/main amd64 Packages [1344 kB]
Get:11 http://archive.ubuntu.com/ubuntu bionic/restricted amd64 Packages [13.5 kB]
Get:12 http://archive.ubuntu.com/ubuntu bionic/multiverse amd64 Packages [186 kB]
Get:13 http://archive.ubuntu.com/ubuntu bionic-updates/restricted amd64 Packages [66.6 kB]
Get:14 http://archive.ubuntu.com/ubuntu bionic-updates/universe amd64 Packages [1376 kB]
Get:15 http://archive.ubuntu.com/ubuntu bionic-updates/multiverse amd64 Packages [19.8 kB]
Get:16 http://archive.ubuntu.com/ubuntu bionic-updates/main amd64 Packages [1205 kB]
Get:17 http://archive.ubuntu.com/ubuntu bionic-backports/main amd64 Packages [8286 B]
Get:18 http://archive.ubuntu.com/ubuntu bionic-backports/universe amd64 Packages [8158 B]
```


Prometheus Setup

Once you run the command, you will see bunch of outputs corresponding to the build steps in Dockerfile. You will also see it downloading the required binaries and tools as follows,

A terminal window titled 'docker-compose run --service-ports prometheus_demo' showing the output of a Docker Compose command. The terminal displays a series of steps for downloading Prometheus binaries. Step 6/12 shows the download of 'node_exporter-0.18.1.linux-amd64.tar.gz' from GitHub. Step 7/12 shows the download of 'alertmanager-0.20.0.linux-amd64.tar.gz' from GitHub. The terminal output is as follows:

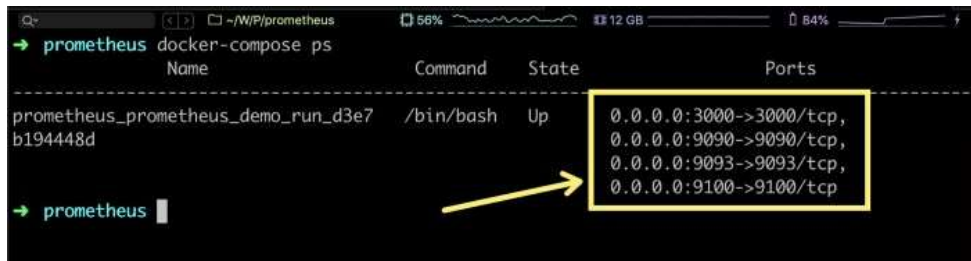
```
2020-05-08 03:07:18 URL:https://github-production-release-asset-2e65be.s3.amazonaws.com/6838921/c0c68480-8f07-11ea-86e0-5d960d0d671d?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=AKI...
Step 6/12 : RUN wget -nv https://github.com/prometheus/node_exporter/releases/download/v0.18.1/node_exporter-0.18.1.linux-amd64.tar.gz
---> Running in 3d6cda9d43d6
2020-05-08 03:07:24 URL:https://github-production-release-asset-2e65be.s3.amazonaws.com/9524057/5dc5df80-86f1-11e9-924c-ef392e7300e3?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=AKI...
Step 7/12 : RUN wget -nv https://github.com/prometheus/alertmanager/releases/download/v0.20.0/alertmanager-0.20.0.linux-amd64.tar.gz
---> Running in f7fe5497faa6
```

Prometheus Setup

Finally, it will drop you in an interactive prompt inside the running container `prometheus_demo_run`.

```
Removing intermediate container a1cd18afe94d
--> 43d8cc190894
Successfully built 43d8cc190894
Successfully tagged prometheus_prometheus_demo:latest
WARNING: Image for service prometheus_demo was built because it did not already exist. To rebuild this image you must use `docker-compose build` or `docker-compose up --build`.
root@452e3f7c04de:~#
```

If you want to see the port mappings of the container to the host, we can go to another terminal keeping this one on (else the running instance will exit because it has nothing to do), and run `docker-compose ps` you will see the status of your container. Right now it's in a bash shell and the state is up and the ports are exposed correctly as shown below,



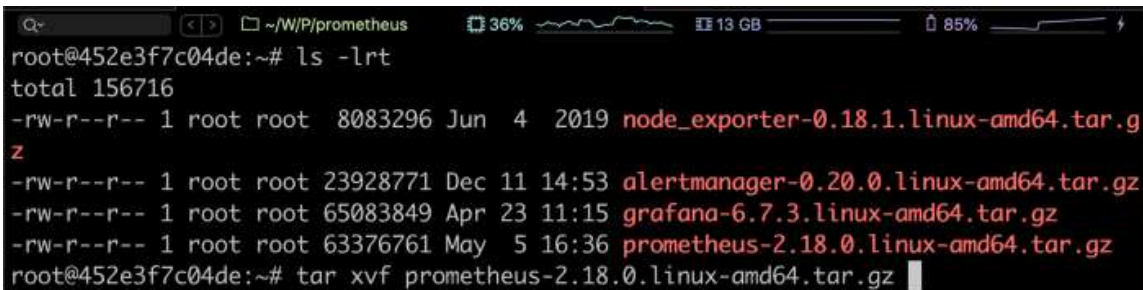
The screenshot shows a terminal window with the command `docker-compose ps` executed. The output is a table with columns: Name, Command, State, and Ports. The container `prometheus_prometheus_demo_run_d3e7b194448d` is shown with state `Up` and ports `0.0.0.0:3000->3000/tcp, 0.0.0.0:9090->9090/tcp, 0.0.0.0:9093->9093/tcp, 0.0.0.0:9100->9100/tcp`. A yellow box highlights the ports column, and a yellow arrow points to it from the left.

Name	Command	State	Ports
prometheus_prometheus_demo_run_d3e7b194448d	/bin/bash	Up	0.0.0.0:3000->3000/tcp, 0.0.0.0:9090->9090/tcp, 0.0.0.0:9093->9093/tcp, 0.0.0.0:9100->9100/tcp

Prometheus Setup

This is all the setup needed, now the fun time begins. If you list the directory you will all our required tools already downloaded for us.

We will first start Prometheus server to get things rolling, you can follow along by simple extracting the package.

A terminal window screenshot showing a directory listing and an extraction command. The window title is '~/.W/P/prometheus'. The terminal output shows the results of 'ls -lrt' and the command 'tar xvf prometheus-2.18.0.linux-amd64.tar.gz'.

```
root@452e3f7c04de:~# ls -lrt
total 156716
-rw-r--r-- 1 root root 8083296 Jun  4 2019 node_exporter-0.18.1.linux-amd64.tar.gz
-rw-r--r-- 1 root root 23928771 Dec 11 14:53 alertmanager-0.20.0.linux-amd64.tar.gz
-rw-r--r-- 1 root root 65083849 Apr 23 11:15 grafana-6.7.3.linux-amd64.tar.gz
-rw-r--r-- 1 root root 63376761 May  5 16:36 prometheus-2.18.0.linux-amd64.tar.gz
root@452e3f7c04de:~# tar xvf prometheus-2.18.0.linux-amd64.tar.gz
```

Prometheus Setup

And running prometheus command inside. It starts up the server with some default configuration. Here you can see it starts off with 15 days of default retention period.

```
root@452e3f7c04de:~# cd prometheus-2.18.0.linux-amd64/  
root@452e3f7c04de:~/prometheus-2.18.0.linux-amd64# ./prometheus  
level=info ts=2020-05-08T03:10:58.980Z caller=main.go:302 msg="No time or size retention was set so using the default time retention" duration=15d  
level=info ts=2020-05-08T03:10:58.980Z caller=main.go:337 msg="Starting Prometheus" version="(version=2.18.0, branch=HEAD, revision=a12e96299dcd159ea09b260f1a21e7e4b86e011d)"  
level=info ts=2020-05-08T03:10:58.980Z caller=main.go:338 build_context="(go=go1.14.2, user=root@7fbcff55abdb, date=20200505-14:26:04)"
```

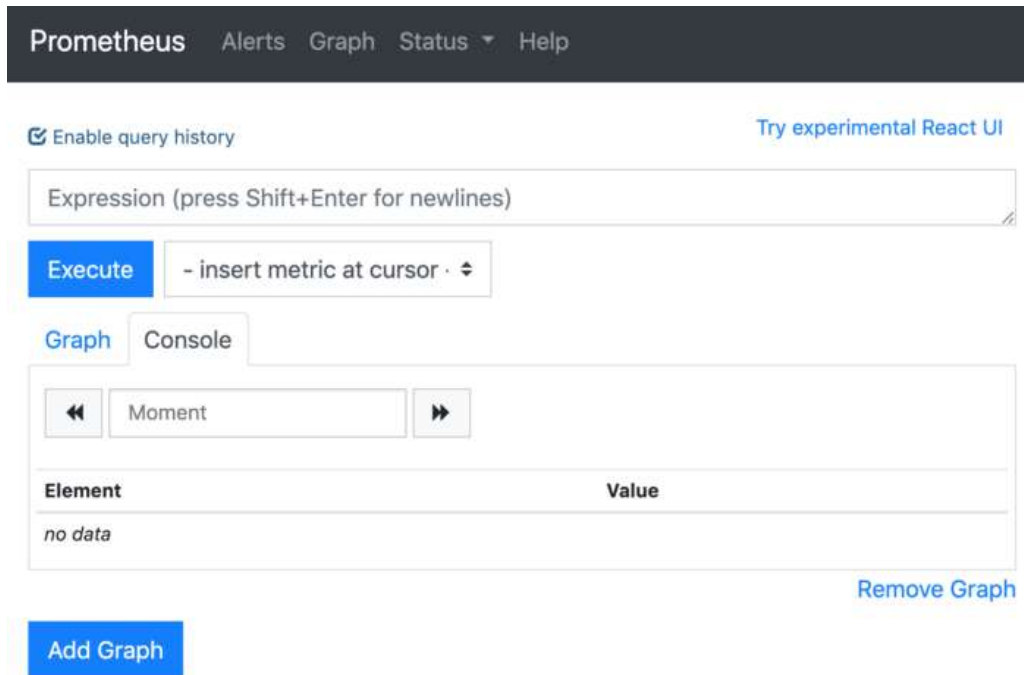
Prometheus Setup

Also, if you notice in the end, **it actually starts the Database with message “TSDB started”** which indicates the time series DB is at the heart of it. Lastly, it reads a file on the same directory called ***prometheus.yml*** which holds the configuration for the server. We will look at it more closely in next sections.

```
level=info ts=2020-05-08T03:10:59.001Z caller=main.go:694 msg="TSDB started"
level=info ts=2020-05-08T03:10:59.001Z caller=main.go:798 msg="Loading configuration
n file" filename=prometheus.yml
level=info ts=2020-05-08T03:10:59.003Z caller=main.go:826 msg="Completed loading of
configuration file" filename=prometheus.yml
level=info ts=2020-05-08T03:10:59.003Z caller=main.go:645 msg="Server is ready to r
eceive web requests."
```

Prometheus Setup

Now, in your host machine (laptop) if you open up the URL `http://localhost:9090/` You will get something like this.



The screenshot displays the Prometheus web interface. At the top is a dark navigation bar with links for Prometheus, Alerts, Graph, Status, and Help. Below this, there's a toggle for 'Enable query history' and a link to 'Try experimental React UI'. The main query editor area contains a text input for the expression, an 'Execute' button, and a dropdown menu for inserting metrics. Below the editor are tabs for 'Graph' and 'Console'. The 'Graph' tab is active, showing a timeline view with a 'Moment' selector and navigation arrows. Below the timeline is a table with two columns: 'Element' and 'Value'. The table currently shows 'no data'. At the bottom right of the graph area is a 'Remove Graph' link, and at the bottom center is an 'Add Graph' button.

Prometheus Alerts Graph Status ▾ Help

☒ Enable query history [Try experimental React UI](#)

Expression (press Shift+Enter for newlines)

Execute - insert metric at cursor ↕

Graph Console

⏪ Moment ⏩

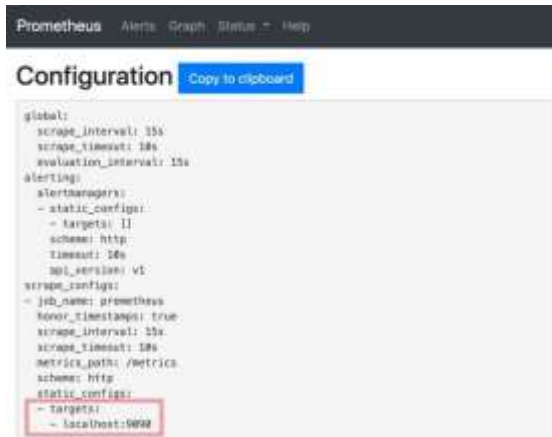
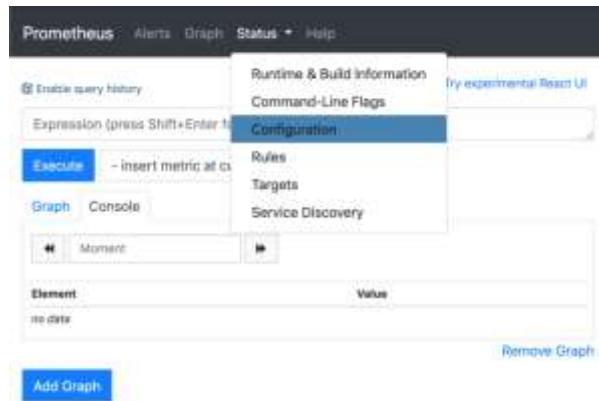
Element	Value
no data	

[Remove Graph](#)

Add Graph

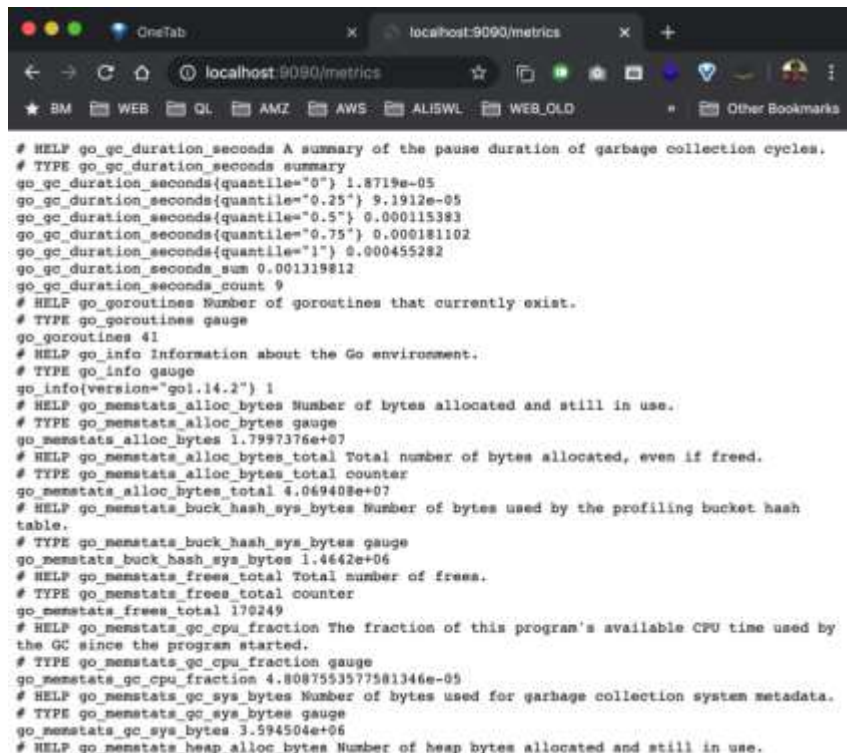
Prometheus Setup

This indicates your **Prometheus server is up and running successfully**, ready to be used. Before we get into any details, I would like to show you the **configuration** with which Prometheus was loaded in the first step.



Prometheus Setup

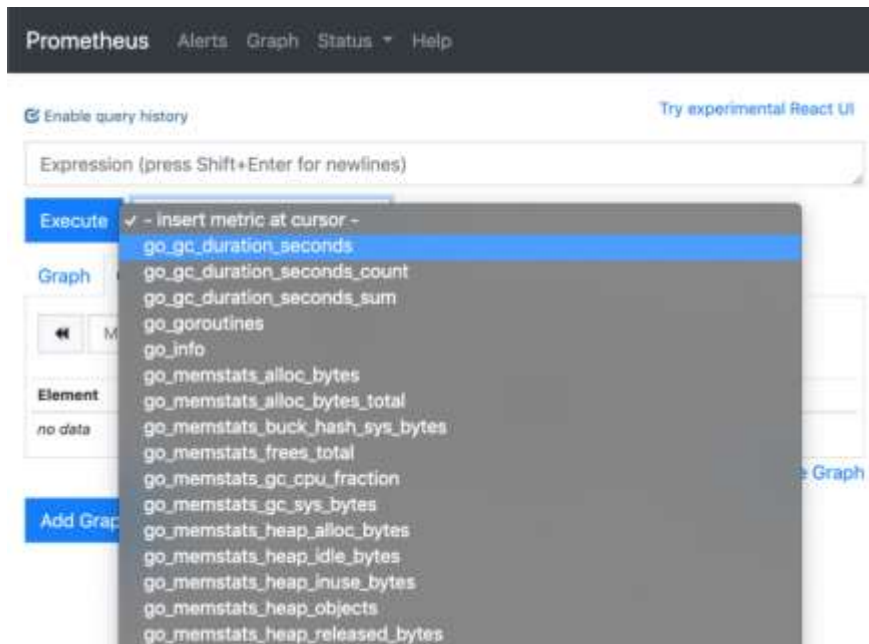
You notice its a very simple configuration like interval of polling (scrape interval), where to poll from etc. Interesting thing to note here is that, Prometheus itself exposes its metrics on the same port in /metrics path for the Prometheus Server to pull data about its own health. Let's see what it is, if you open up <http://localhost:9090/metrics> sure enough you will simple key/values with some comments.

A screenshot of a web browser window displaying the Prometheus /metrics endpoint. The browser's address bar shows 'localhost:9090/metrics'. The page content is a text-based output of various Go metrics, including garbage collection statistics, goroutine counts, and memory usage details. The metrics are presented in a structured format with comments, types, and numerical values.

```
# HELP go_gc_duration_seconds A summary of the pause duration of garbage collection cycles.
# TYPE go_gc_duration_seconds summary
go_gc_duration_seconds{quantile="0"} 1.8719e-05
go_gc_duration_seconds{quantile="0.25"} 9.1912e-05
go_gc_duration_seconds{quantile="0.5"} 0.000113383
go_gc_duration_seconds{quantile="0.75"} 0.000181102
go_gc_duration_seconds{quantile="1"} 0.000455282
go_gc_duration_seconds_sum 0.001339812
go_gc_duration_seconds_count 9
# HELP go_goroutines Number of goroutines that currently exist.
# TYPE go_goroutines gauge
go_goroutines 41
# HELP go_info Information about the Go environment.
# TYPE go_info gauge
go_info{version="go1.14.2"} 1
# HELP go_memstats_alloc_bytes Number of bytes allocated and still in use.
# TYPE go_memstats_alloc_bytes gauge
go_memstats_alloc_bytes 1.7997376e+07
# HELP go_memstats_alloc_bytes_total Total number of bytes allocated, even if freed.
# TYPE go_memstats_alloc_bytes_total counter
go_memstats_alloc_bytes_total 4.069408e+07
# HELP go_memstats_buck_hash_sys_bytes Number of bytes used by the profiling bucket hash table.
# TYPE go_memstats_buck_hash_sys_bytes gauge
go_memstats_buck_hash_sys_bytes 1.4642e+06
# HELP go_memstats_frees_total Total number of frees.
# TYPE go_memstats_frees_total counter
go_memstats_frees_total 170249
# HELP go_memstats_gc_cpu_fraction The fraction of this program's available CPU time used by the GC since the program started.
# TYPE go_memstats_gc_cpu_fraction gauge
go_memstats_gc_cpu_fraction 4.808753577581346e-05
# HELP go_memstats_gc_sys_bytes Number of bytes used for garbage collection system metadata.
# TYPE go_memstats_gc_sys_bytes gauge
go_memstats_gc_sys_bytes 3.594504e+06
# HELP go_memstats_heap_alloc_bytes Number of heap bytes allocated and still in use.
```


Prometheus Setup

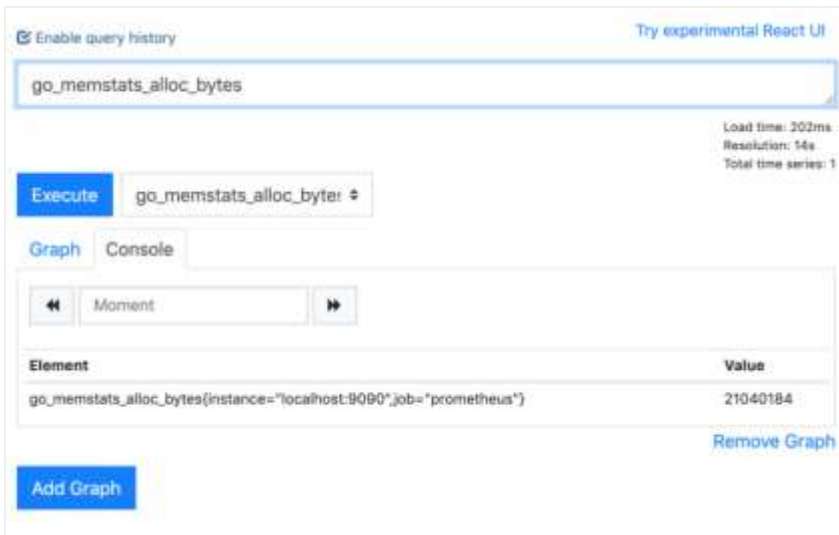
But essentially it means, we already have metrics that we are collecting and can be queried from Prometheus Server, so let's get back to the server. If you click on the available metrics dropdown, sure enough all those metrics are there.



Prometheus Setup

Let's select a metric to explore little more, let's say we want to know how much memory is allocated for Prometheus server, we select `go_memstats_alloc_bytes` (Just an example, you can use whatever you want).

If I execute this query you will see, I get one value.
By default, any metric query gives the last recorded result.



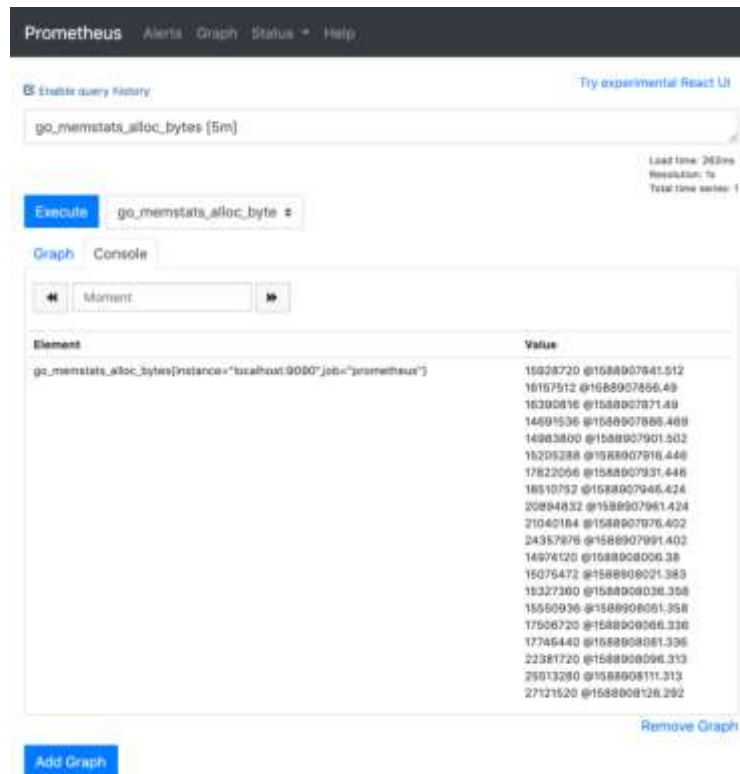
The screenshot shows the Prometheus web interface. At the top, there's a checkbox for "Enable query history" and a link for "Try experimental React UI". The query input field contains `go_memstats_alloc_bytes`. To the right of the input, it shows "Load time: 202ms", "Resolution: 14s", and "Total time series: 1". Below the input is a blue "Execute" button and a text field containing `go_memstats_alloc_bytes`. There are two tabs: "Graph" (selected) and "Console". Below the tabs is a time range selector with a "Moment" button and left/right arrows. A table displays the query results:

Element	Value
<code>go_memstats_alloc_bytes(instance="localhost:9090",job="prometheus")</code>	21040184

At the bottom right of the table is a blue link "Remove Graph". At the bottom left is a blue button "Add Graph".

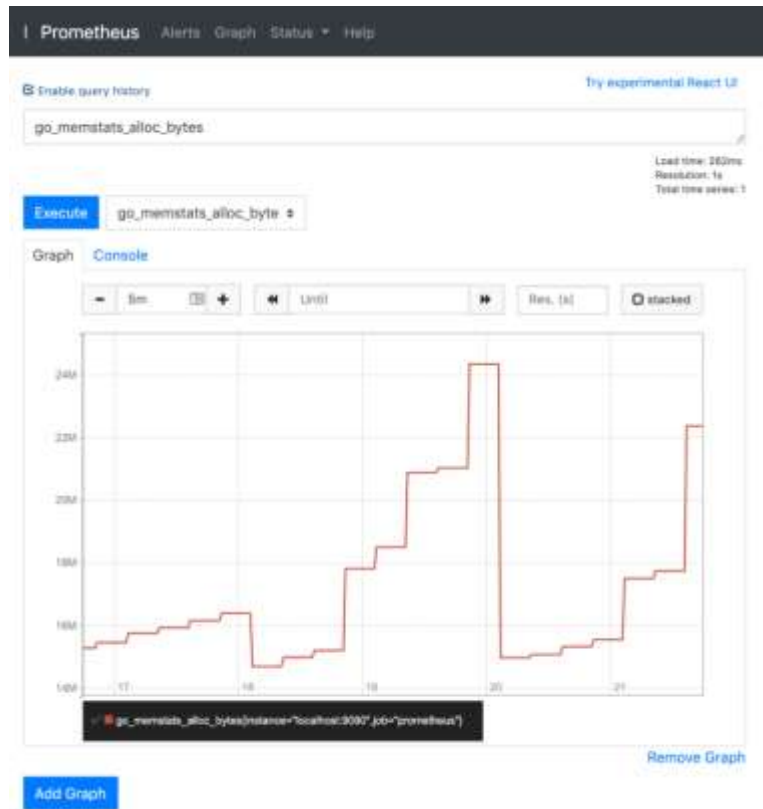
Prometheus Setup

But, if we want to know how this has changed over time, we can use the PromQL query language to do so as shown in the example, its simple and expressive language in fact this is as powerful as the Time Series DB itself and Prometheus is nothing but these two together. More on this.



Prometheus Setup

Now, we get all the values recorded in last 5 minutes and you can see all of them have a value and a timestamp associated with it. You can also visualize the data in the visualize tab. With this you can clearly see, **how the memory consumption is happening for Prometheus server.**



Exporters in Prometheus

There are a number of libraries and servers which help in exporting existing metrics from third-party systems as Prometheus metrics. This is useful for cases where it is not feasible to instrument a given system with Prometheus metrics directly (for example, MySQL Server, HAProxy or Linux system stats).

Prometheus has huge ecosystem of exporters, almost any popular software that you want to use it. We will look at one of official ones in details [here](#)

Node Exporter (Machine Health)

Node exporter is a simple health metric exporter for your system. It collects many many system level metrics by looking at the /proc files of your system. We have already downloaded node exporter in our setup step, to understand more let's fire it up in our Docker shell (new shell using screen)

```
root@d925f474f53f:~# tar -xvf node_exporter-0.18.1.linux-amd64.tar.gz
node_exporter-0.18.1.linux-amd64/
node_exporter-0.18.1.linux-amd64/node_exporter
node_exporter-0.18.1.linux-amd64/NOTICE
node_exporter-0.18.1.linux-amd64/LICENSE
root@d925f474f53f:~#
```

Node Exporter (Machine Health)

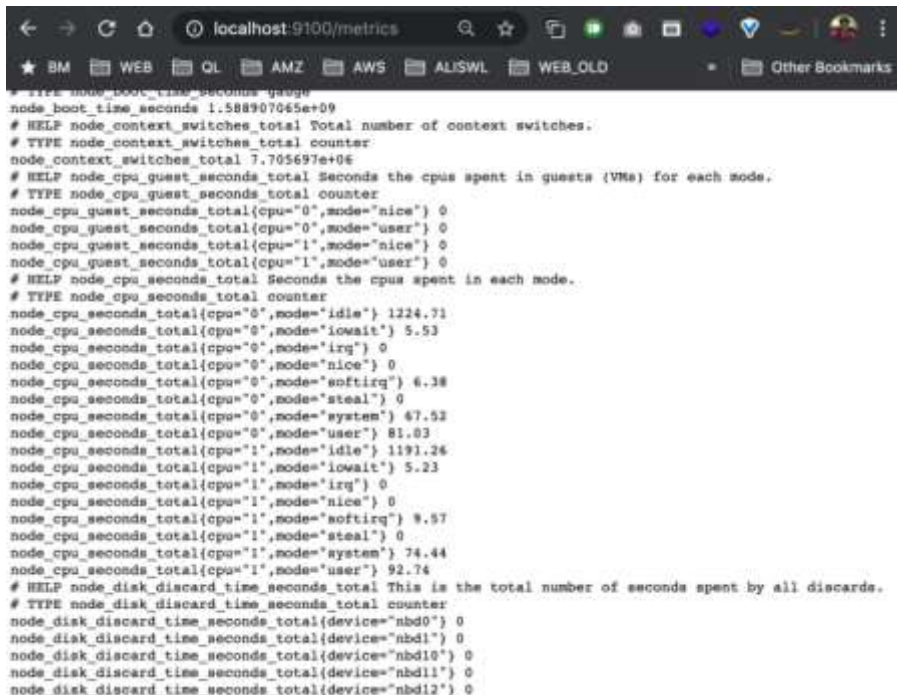
You can see once we start the node exporter, it starts the collectors for many many individual stats and exposes them to be consumed at the port 9100.

```
root@d925f474f53f:~/node_exporter-0.18.1.linux-amd64# ./node_exporter
INFO[0000] Starting node_exporter (version=0.18.1, branch=HEAD, revision=3db77732e925c08f675d7404a8c46466b2ece83e) source="node_exporter.go:156"
INFO[0000] Build context (go=go1.12.5, user=root@b50852a1acba, date=20190604-16:41:18) source="node_exporter.go:157"
INFO[0000] Enabled collectors: source="node_exporter.go:97"
INFO[0000] - arp source="node_exporter.go:104"
INFO[0000] - bcache source="node_exporter.go:104"
INFO[0000] - bonding source="node_exporter.go:104"
INFO[0000] - conntrack source="node_exporter.go:104"
INFO[0000] - cpu source="node_exporter.go:104"
INFO[0000] - cpufreq source="node_exporter.go:104"
```

```
INFO[0000] - timex source="node_exporter.go:104"
INFO[0000] - uname source="node_exporter.go:104"
INFO[0000] - vmstat source="node_exporter.go:104"
INFO[0000] - xfs source="node_exporter.go:104"
INFO[0000] - zfs source="node_exporter.go:104"
INFO[0000] Listening on :9100 source="node_exporter.go:170"
```

Node Exporter (Machine Health)

If you go to the URL <http://localhost:9100/metrics> you will see a very similar kind of metrics page we saw earlier with Prometheus server metrics.



```
* TYPE node_boot_time_seconds gauge
node_boot_time_seconds 1.588907065e+09
# HELP node_context_switches_total Total number of context switches.
# TYPE node_context_switches_total counter
node_context_switches_total 7.705697e+06
# HELP node_cpu_guest_seconds_total Seconds the cpus spent in guests (VMs) for each node.
# TYPE node_cpu_guest_seconds_total counter
node_cpu_guest_seconds_total{cpu="0",mode="nice"} 0
node_cpu_guest_seconds_total{cpu="0",mode="user"} 0
node_cpu_guest_seconds_total{cpu="1",mode="nice"} 0
node_cpu_guest_seconds_total{cpu="1",mode="user"} 0
# HELP node_cpu_seconds_total Seconds the cpus spent in each mode.
# TYPE node_cpu_seconds_total counter
node_cpu_seconds_total{cpu="0",mode="idle"} 1224.71
node_cpu_seconds_total{cpu="0",mode="iowait"} 5.53
node_cpu_seconds_total{cpu="0",mode="irq"} 0
node_cpu_seconds_total{cpu="0",mode="nice"} 0
node_cpu_seconds_total{cpu="0",mode="softirq"} 6.38
node_cpu_seconds_total{cpu="0",mode="steal"} 0
node_cpu_seconds_total{cpu="0",mode="system"} 47.52
node_cpu_seconds_total{cpu="0",mode="user"} 81.03
node_cpu_seconds_total{cpu="1",mode="idle"} 1191.26
node_cpu_seconds_total{cpu="1",mode="iowait"} 5.23
node_cpu_seconds_total{cpu="1",mode="irq"} 0
node_cpu_seconds_total{cpu="1",mode="nice"} 0
node_cpu_seconds_total{cpu="1",mode="softirq"} 9.57
node_cpu_seconds_total{cpu="1",mode="steal"} 0
node_cpu_seconds_total{cpu="1",mode="system"} 74.44
node_cpu_seconds_total{cpu="1",mode="user"} 92.74
# HELP node_disk_discard_time_seconds_total This is the total number of seconds spent by all discards.
# TYPE node_disk_discard_time_seconds_total counter
node_disk_discard_time_seconds_total{device="nbd0"} 0
node_disk_discard_time_seconds_total{device="nbd1"} 0
node_disk_discard_time_seconds_total{device="nbd10"} 0
node_disk_discard_time_seconds_total{device="nbd11"} 0
node_disk_discard_time_seconds_total{device="nbd12"} 0
```


Node Exporter (Machine Health)

This time as you can see if has the keys start with node_ which differentiates the key space.

Now the next step is how do it collect these metrics in Prometheus server. For that, we will stop our Prometheus server (I am using screen to open multiple shells in my session) and edit our config file like this,

```
# metrics_path defaults to '/metrics'
# scheme defaults to 'http'.

static_configs:
- targets: ['localhost:9090','localhost:9100']

~
"prometheus.yml" 29L, 943C                                27,0-1          Bot
```

Node Exporter (Machine Health)

We added one more target for our server, now the server will poll both of these targets at the specified time interval to collect metrics.

With this we start the Prometheus server and verify the config is loaded successfully

Configuration

[Copy to clipboard](#)

```
global:
  scrape_interval: 15s
  scrape_timeout: 10s
  evaluation_interval: 15s
alerting:
  alertmanagers:
    - static_configs:
      - targets: []
      scheme: http
      timeout: 10s
      api_version: v1
scrape_configs:
  - job_name: prometheus
    honor_timestamps: true
    scrape_interval: 15s
    scrape_timeout: 10s
    metrics_path: /metrics
    scheme: http
    static_configs:
      - targets:
        - localhost:9090
        - localhost:9100
```

Metrics Path

Target Locations where metrics are available

You can see now in the targets section we have two targets as expected. In next sections we will explore more on Queries and Visualisation based on this node exporter data only.

Prometheus Queries

The whole reason we opted for a Time Series DB like Prometheus is so that we can query the data in a very flexible way. And Prometheus lives upto this promise with it's powerful PromQL language. Let's take a look at a simple metric first

The screenshot shows the Prometheus web interface. At the top is a navigation bar with links for Prometheus, Alerts, Graph, Status, and Help. Below this is a sub-header with a checkbox for 'Enable query history' and a link to 'Try experimental React UI'. The main query input field contains 'node_cpu_seconds_total'. To the right of the input field, performance metrics are displayed: 'Load time: 282ms', 'Resolution: 14s', and 'Total time series: 16'. Below the input field is an 'Execute' button and a dropdown menu showing the selected query. The 'Graph' tab is active, showing a table of data points for the query. The table has two columns: 'Element' and 'Value'. The data points represent different CPU modes for the 'node_cpu_seconds_total' metric on a specific instance.

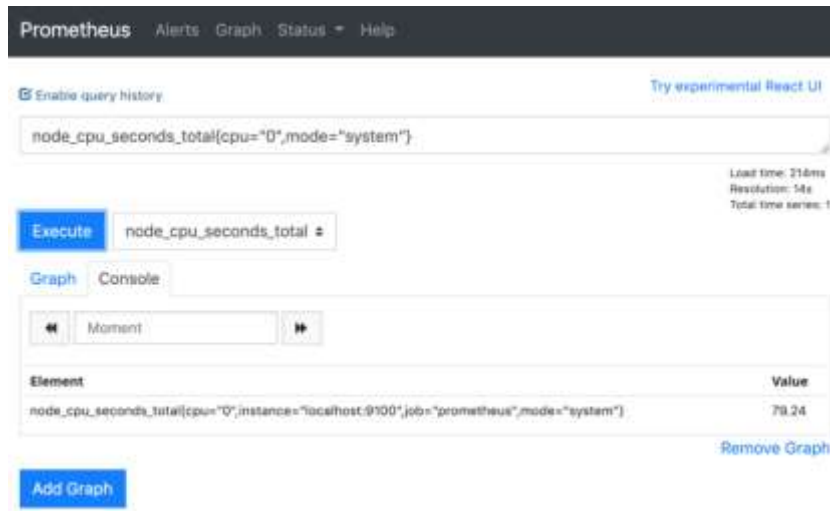
Element	Value
node_cpu_seconds_total{cpu="0",instance="localhost:9100",job="prometheus",mode="idle"}	1538.1
node_cpu_seconds_total{cpu="0",instance="localhost:9100",job="prometheus",mode="iowait"}	6.18
node_cpu_seconds_total{cpu="0",instance="localhost:9100",job="prometheus",mode="irq"}	0
node_cpu_seconds_total{cpu="0",instance="localhost:9100",job="prometheus",mode="nice"}	0
node_cpu_seconds_total{cpu="0",instance="localhost:9100",job="prometheus",mode="softirq"}	7.16
node_cpu_seconds_total{cpu="0",instance="localhost:9100",job="prometheus",mode="steal"}	0
node_cpu_seconds_total{cpu="0",instance="localhost:9100",job="prometheus",mode="system"}	78.23
node_cpu_seconds_total{cpu="0",instance="localhost:9100",job="prometheus",mode="user"}	92.08
node_cpu_seconds_total{cpu="1",instance="localhost:9100",job="prometheus",mode="idle"}	1480.1
node_cpu_seconds_total{cpu="1",instance="localhost:9100",job="prometheus",mode="iowait"}	5.83

Prometheus Queries

You can see with node exporter, it collects nested info as to which is the host and instance sending this data.

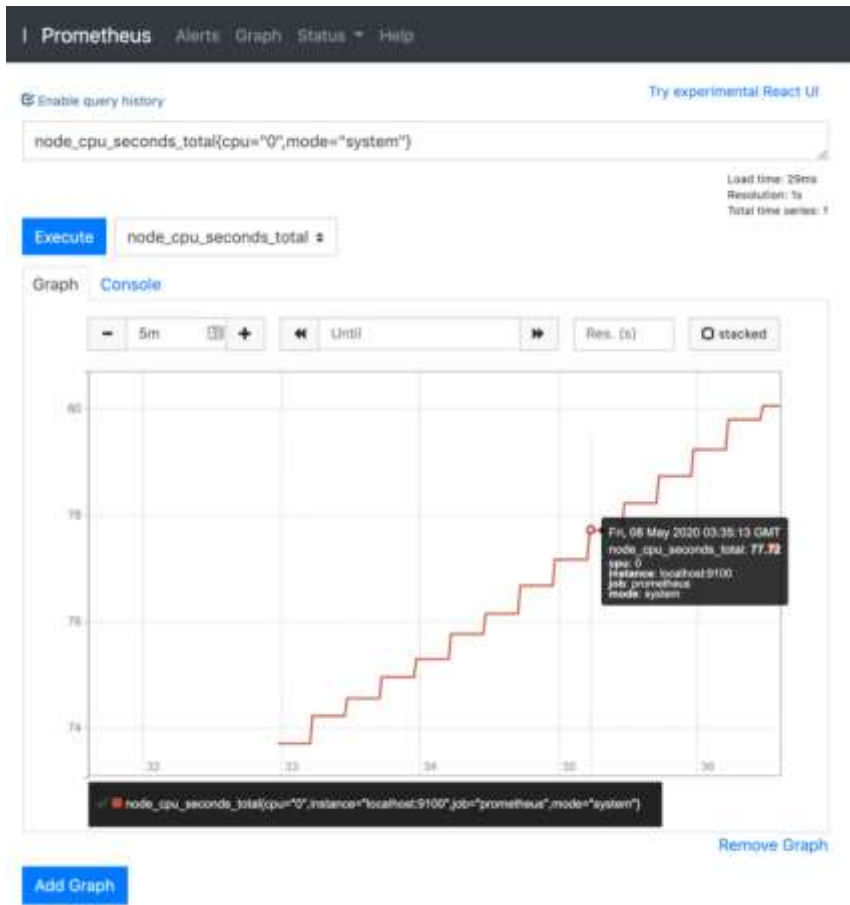
One important thing to note here is that if you add more nodes and export data using node exporter, the metrics will have the same keys but will differ in the nested context like above where the instance or cpu will be different. So in Prometheus metrics are collected by service and not my host. Very important to keep that in mind.

We can add more conditions to the query for drilling down to a specific metric like this, it gives us the latest recorded value for the metric



Prometheus Queries

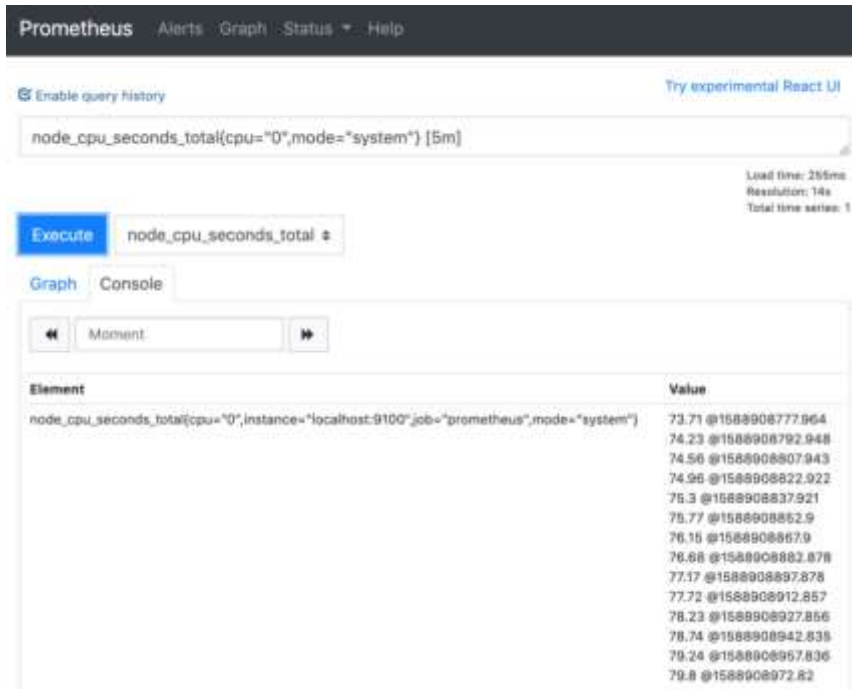
We can also visualize the same data in Graph tab which shows the data collected over time



Prometheus Queries

If you want to see the actual data points, we can add [5m] time frame to our query and execute the same, here are the values for the last 5m interval shown in the diagram below.

So, with this simple setup you can see how much you can get for free without doing pretty much no work. Prometheus also supports publishing Custom metrics as well in a very powerful way, in all possible languages. Huge community again thrives on the aspect of choices and flexibility to mix and match your solutions.



Grafana

Now that we have a Prometheus server collecting metrics for us, we would want a very powerful visualisation tool which can be used for all sorts of dashboard and monitoring requirements.

Grafana is a great tool in this regard. It integrates with almost all popular sources of metrics server like Prometheus and allows us much more visual tooling which ultimately converts them to complex queries to Prometheus server, but hey! we don't have to worry about them.

So, let's extract the downloaded Grafana in a new shell (use Screen to open a new shell Ctrl+A+C) and start the grafana-server as shown below

```
root@d925f474f53f:~/grafana-6.7.3# ls
LICENSE NOTICE.md README.md VERSION bin conf public scripts tools
root@d925f474f53f:~/grafana-6.7.3# cd bin/
root@d925f474f53f:~/grafana-6.7.3/bin# ls
grafana-cli grafana-cli.md5 grafana-server grafana-server.md5
root@d925f474f53f:~/grafana-6.7.3/bin# ./grafana-server
```

Grafana

It will start the Grafana server on default port 3000.

```
INFO[05-08|03:40:01] Initializing UsageStatsService      logger=server
INFO[05-08|03:40:01] Initializing CleanupService         logger=server
INFO[05-08|03:40:01] Initializing NotificationService    logger=server
INFO[05-08|03:40:01] Initializing provisioningServiceImpl logger=server
INFO[05-08|03:40:01] Initializing Stream Manager
INFO[05-08|03:40:01] HTTP Server Listen                  logger=http.server address=[::]:3000 protocol=http s
ubUrl= socket=
INFO[05-08|03:40:01] Backend rendering via phantomJS      logger=rendering renderer=phantomJS
WARN[05-08|03:40:01] phantomJS is deprecated and will be removed in a future release. You should consider migratin
g from phantomJS to grafana-image-renderer plugin. Read more at https://grafana.com/docs/grafana/latest/administra
tion/image_rendering/ logger=rendering renderer=phantomJS
```

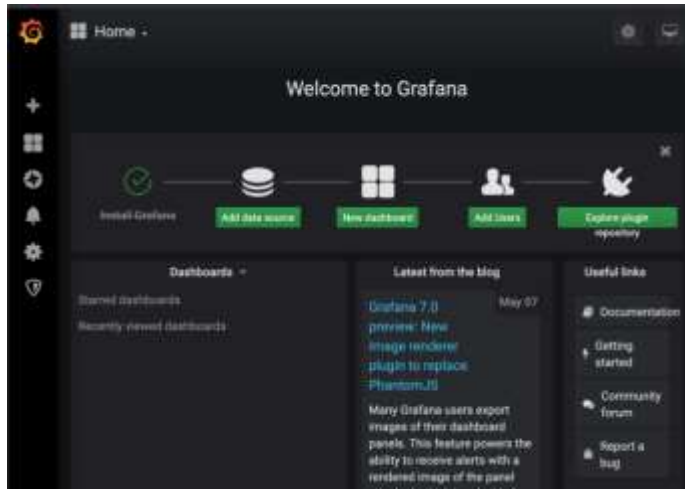

Grafana

If we go to `http://localhost:3000` we will get to the grafana homepage. We can login to grafana with default username admin and password admin. (For first time users it might ask you change the password initially, feel free to set it up they way you like it)

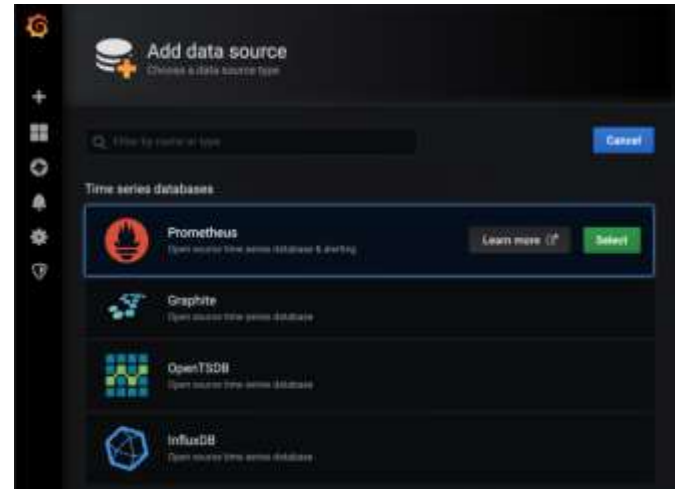


Grafana

Once logged in you will see, first thing Grafana needs is some data source to work with. In our case, we will go ahead and click on the add data source link.

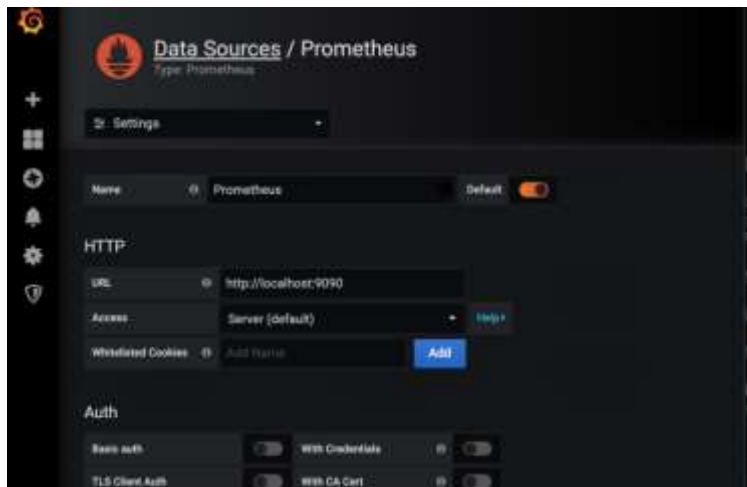


In the options you can see Grafana is capable of talking to multiple Databases, Prometheus being on the top :)

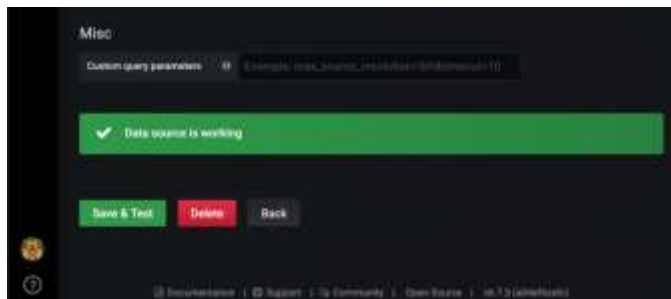


Grafana

So we select Prometheus and provide the URL of our Prometheus Server.

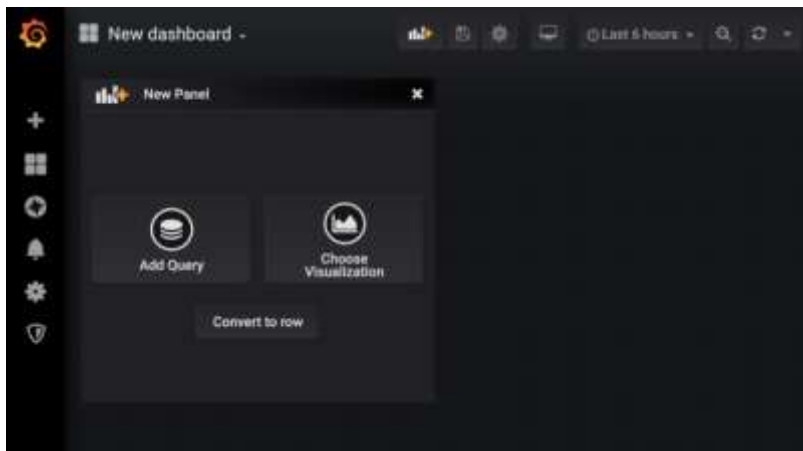


We can save and test the configuration to make sure the data source is working properly.

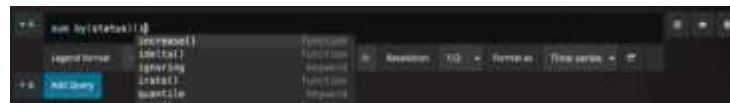
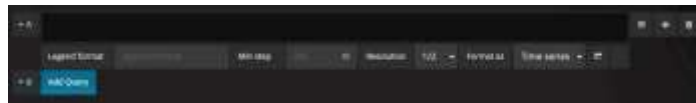


Grafana

With that setup out of the way, we can now use Grafana to create dashboards and Alarms (yes Alarms can be done in Grafana layer itself, though we will show you alertmanager which works in tandem with Prometheus)

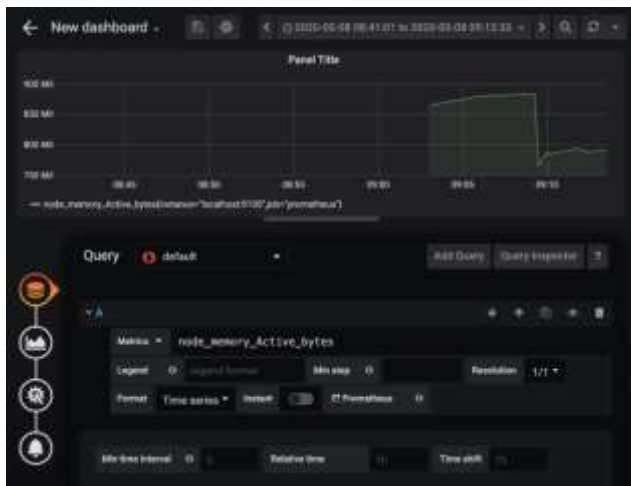


For fun, let's add a Query for memory usage of the whole node. Grafana has very powerful autocomplete feature, just to illustrate, here is how you get autocomplete in the UI,



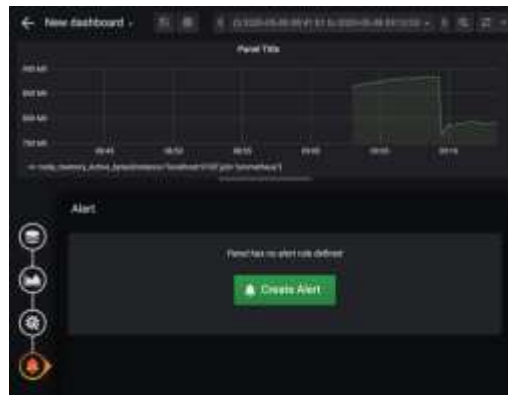
Grafana

So, if we start typing, node_ we will get a bunch of auto completes, for now, we will use the metric `node_memory_Active_bytes` to see the memory usage.



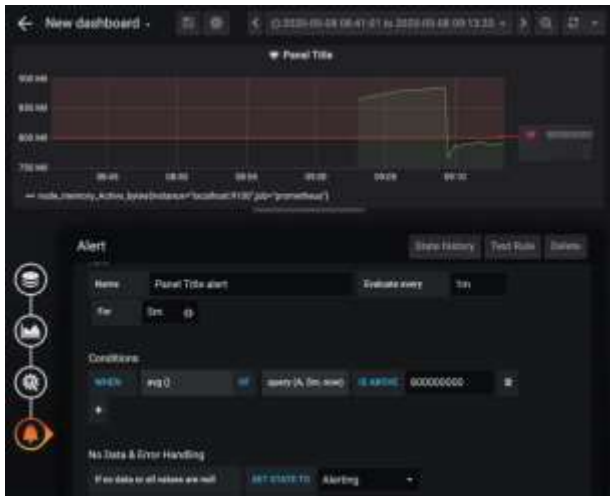
Immediately you can see data starts to flow in in a nice graph. Give yourself a round of applause if everything worked out so far.

But, this is not the end as we said, with Grafana we can add Alarms right where graphs are configured. Let's do one for fun, so we go to add alarms



Grafana

And configure the alarm conditions, thresholds, you can see the alarm indicator comes alive showing you exactly where the alarm is getting set to.

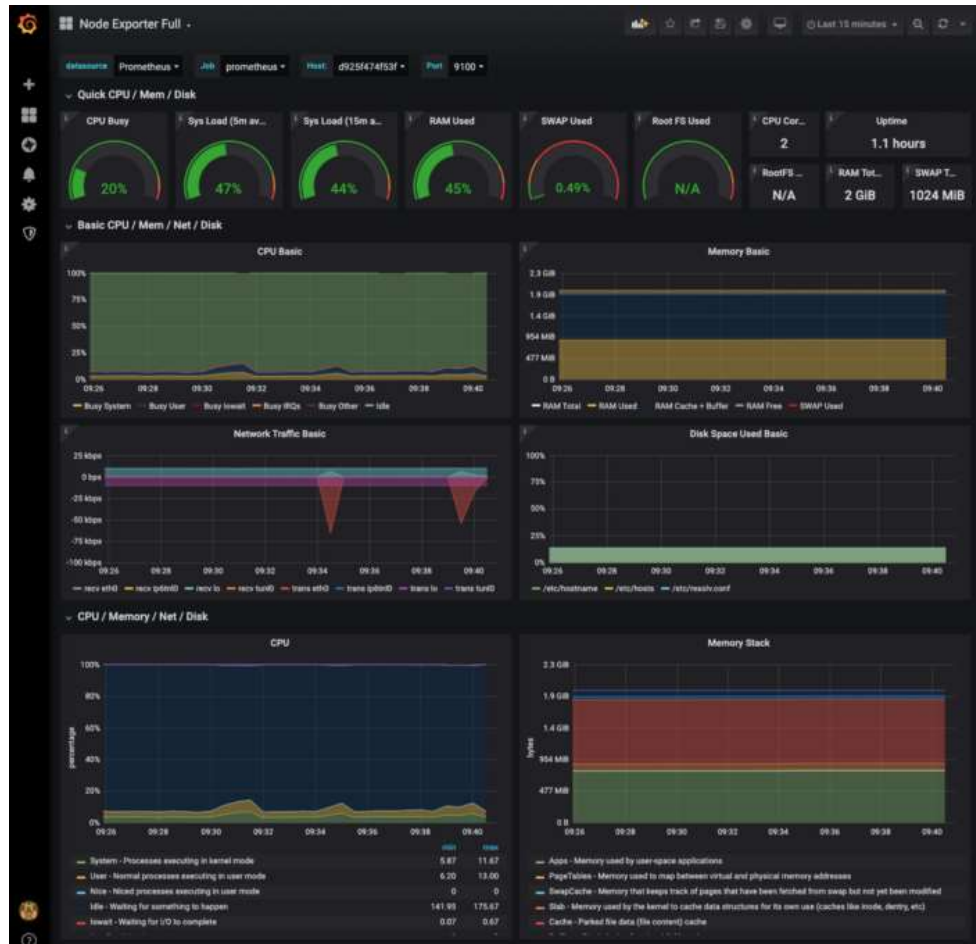


What we learnt is just tip of the iceberg on what you can do with Grafana. For example, in Grafana dashboard can be exported and imported in JSON format (Completely sharable). And again, there is a huge community who has already shared a lot of dashboard which we can reuse to learn and tweak to our needs.

In this case, for this simple setup, if we want to see the full visualisation in comprehensive dashboard, I have used Node Exporter Full community dashboard.

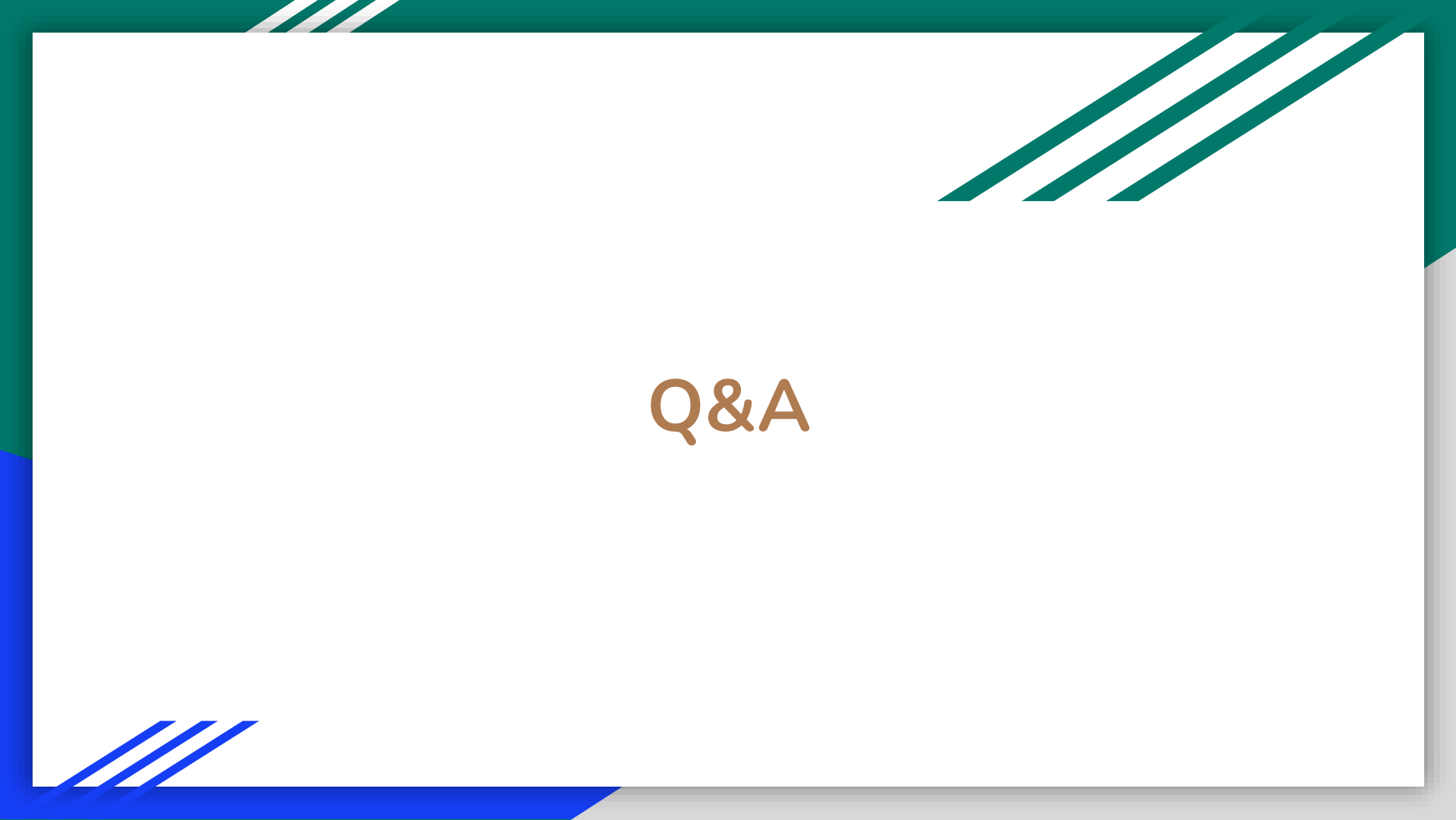
And viola!! This is what we get from it by default, Amazing isn't it :)

Grafana



Conclusion

So, we learned several things, including what Prometheus is and how it works, how to set it up, how we can get metrics from exporters and query them. How we can visualize the same in Grafana.



Q&A